C++

📘 Theory

1. Constructors

- **Default constructor** → no parameters, initializes default values.
- **Parameterized constructor** → accepts arguments to initialize fields.
- Called automatically when object is created.

2. Inheritance

- **Single inheritance** → class Child : public Parent.
- **Multilevel inheritance** → Child2 extends Child which extends Parent.
- **Multiple inheritance** → Child3 : public Parent, public Guardian.

3. Polymorphism

- **Compile-time (overloading)** → same method name, different parameters (add(int,int) vs add(int,int,int)).
- **Runtime (overriding)** → child class redefines parent method (UPI.pay() overrides Payment.pay()).

4. Encapsulation

- Private data (amount in Bank) hidden from outside.
- Accessed via **getter** and **setter** methods.

5. Strings and Math

- string class → supports length, concatenation, indexing.
- <cmath> → functions like sqrt(), ceil(), floor(), pow().

📝 Coding Round Questions

- Constructors → What is the difference between default and parameterized constructors?
- Inheritance → Explain single, multilevel, and multiple inheritance with examples.

- Polymorphism → What is the difference between overloading and overriding?
- Encapsulation → How does encapsulation protect data in C++?
- Access specifiers → Explain public, private, and protected.
- Strings → How do you work with strings in C++?

⚡ **Machine Coding Round Challenge**

Problem: Banking System with OOP

Requirements:

1. Create a BankAccount class with private fields: accountNumber, balance.
2. Add constructors (default and parameterized).
3. Add methods: deposit(), withdraw(), getBalance().
4. Create subclasses SavingsAccount and CurrentAccount that override withdraw() rules.
5. Demonstrate polymorphism by calling methods via base class pointer.

```cpp
#include <iostream>
#include <cmath>
#include <string>
using namespace std;


class Student{
    public:
        int rollNo ;

        Student(){ // default constructor
            rollNo = 21;
            cout<<"Constructor is called"<<endl;
        }

        Student(int roll){ // parameterized constructor
            rollNo = roll;
        }

        void display(){
            cout<<"Display functions"<<endl;
        }
```

```
};

class Guardian{
    public:
        void greet(){
            cout<<"Guardian"<<endl;
        }
};

class Parent{
    public:
        void salary(){
            cout<<"10000"<<endl;
        }
};



class Child:public Parent{

};

class Child2:public Child{

};

class Child3: public Parent, public Guardian{

};

class Arithmetic{
    public:
        void add(int a, int b){
            cout<<a+b<<endl;
        }

        void add(int a, int b,int c){
            cout<<a+b+c<<endl;
        }
};

class Payment{
    public:
        void pay(){
            cout<<"Payment paid"<<endl;
        }
};
```

```cpp
class UPI : public Payment{
    public:
        void pay(){
            cout<<"Payment paid using UPI"<<endl;
        }

};
class COD : public Payment{
    public:
        void pay(){
            cout<<"Payment will be paid using COD"<<endl;
        }
};

class Bank{
    private:
        int amount = 40000;

    public:
        void getAmount(){
            cout<<amount<<endl;
        }
        void setAmount(int amt){
            amount = amt;
        }
};


int main(){

    Bank bank;
    bank.setAmount(50000);
    bank.getAmount();
    // cout<<bank.amount<<endl;
    // bank.display();
    COD cod;
    cod.pay();
    UPI upi;
    upi.pay();


    Arithmetic arithmetic;
    arithmetic.add(10,20);
    arithmetic.add(10,20,30);


    Child3 multiple;
    multiple.greet();
    multiple.salary();
```

```
    Child2 inherittt;
    inherittt.salary();


    Child inherit;
    inherit.salary();



    Student Manisha(15);

    Student Harini(40);
    Student Subhiksha(60);

    cout<<Manisha.rollNo<<endl;

    cout<<Harini.rollNo<<endl;

    cout<<Subhiksha.rollNo<<endl;
    // cout<<"Hello world"<<endl;

    // cout<<sqrt(64)<<endl;
    // cout<<ceil(4.67)<<endl;
    // cout<<floor(4.67)<<endl;
    // cout<<pow(2,7)<<endl;

    string name = "Fi it";

    // cout<<name<<endl;

    // cout<<name[2]<<endl;

    // cout<<name.length()<<endl;

    // string str1 ="Hello";
    // string str2 = "World";

    // string str3 = str1 +" " + str2;
    // cout<<str3<<endl;


    // oops => object oriented programming


    return 0;
}
```